OliveTrace

Processus DevOps

Dossier de validation finale

Applications Web Avancées · 22 octobre 2026

Un projet intégré, des contrôles automatisés et une version reproductible.

Traçabilité de l'huile d'olive tunisienne, de l'origine agricole aux retours consommateurs. Ce dossier présente le processus, les preuves et l'état réel de la livraison.

240 tests · **75,2** % de couverture globale · **2,0** % de duplication

CI VALIDÉE | DOCKER LOCAL VALIDÉ | CD PUBLIC À FINALISER

Sofiene Zayati	Producteurs et fermes
Mariem Ammari	Récolte et moulin
Oussema Benjemia	Laboratoire et certification
Hana Fkiri	Distribution
Aymen Jenhani	Consommateurs

LIRE LE DOSSIER

02 Processus et Git | 03 Pipeline CI/CD | 04 Preuves CI

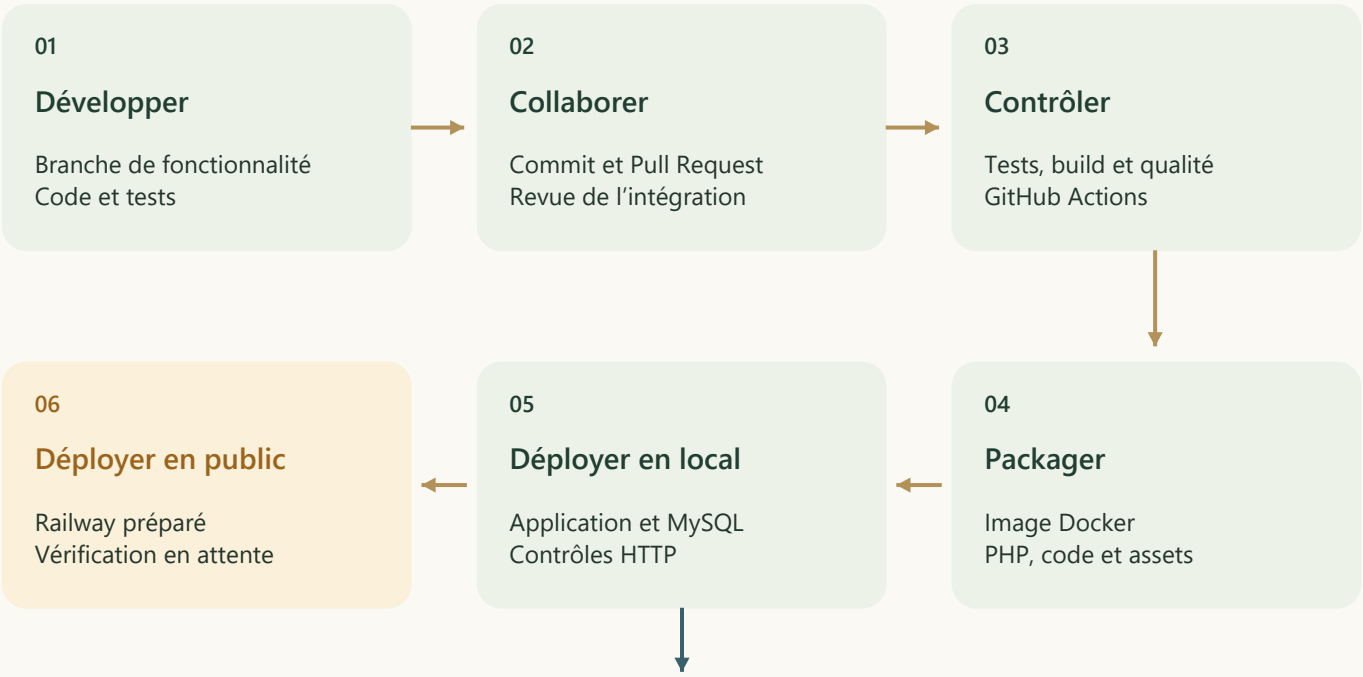
05 Couverture et qualité | 06 Architecture Docker | 07 Déploiement local

08 CD et Railway | 09 Exploitation | 10 Validation

Référence technique mesurée : **5e84736**. Les résultats et captures sont datés ; le déploiement public n'est pas présenté comme achevé.

01 Du développement à la livraison

Le processus relie le travail des cinq membres à une version contrôlée et reproductible.



Vérifié

Préparé / cloud en attente

Organisation du dépôt

feature/* → Pull Request → develop → revue / intégration → main

- **main** est la référence intégrée et la branche prévue pour livrer l'application.
- **develop** est la branche d'intégration. Les PR et les push vers main/develop déclenchent la CI.
- La revue et la promotion restent humaines ; les contrôles de chaque version sont automatisés.

Preuves : historique du dépôt · PR DevOps #8 · workflow de CI

02 Le pipeline, étape par étape

CI = intégration continue. CD = livraison / déploiement après les contrôles de la version stable.

Étape	Exécution réelle	Critère de passage
1 · Installer	PHP 8.4, Composer 2 et Node 24 ; dépendances à partir des fichiers lock.	Composer valide le manifeste et les extensions.
2 · Préparer	MySQL 8.4 jetable en CI ; migrations et données de test.	Base de test distincte des bases locales et hébergées.
3 · Construire	Compilation Vite ; contrôle du format PHP avec Pint.	Assets compilés et formatage conforme.
4 · Tester	Même suite sur SQLite et MySQL ; rapports Clover et JUnit.	240 tests / 1 740 assertions réussis pour chaque base.
5 · Analyser	SonarQube importe les résultats ; attente du Quality Gate.	Un gate en échec fait échouer le job qualité.
6 · Packager	Construction Docker, migration et démarrage du conteneur.	Santé et pages publiques accessibles ; chemins privés refusés.

Deux jobs dépendants

Tests and code quality doit réussir avant **Docker build and HTTP smoke check**. Un échec interrompt la chaîne.

CD public : chaîne prévue

main avec CI verte → Railway attend la CI → build Docker → migration → démarrage → /up sain → routage HTTPS. La connexion Railway et la première release restent à finaliser.

Rapports conservés 14 jours dans les artefacts GitHub Actions. Les appels IA et e-mail sont simulés pendant les tests automatisés.

03 La preuve des contrôles automatisés

Une exécution complète sur main vérifie le code, puis l'image de déploiement.

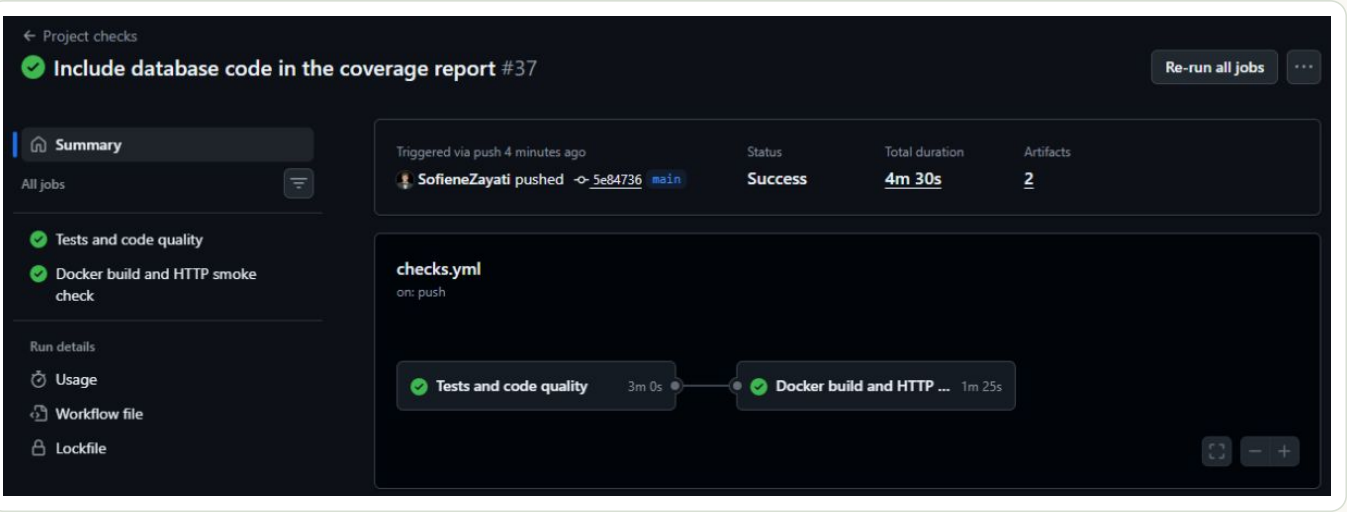


Figure 1. Exécution 5e84736 : les deux jobs sont verts. Le workflow a duré 4 min 30 s.

Ouvrir cette exécution et ses artefacts

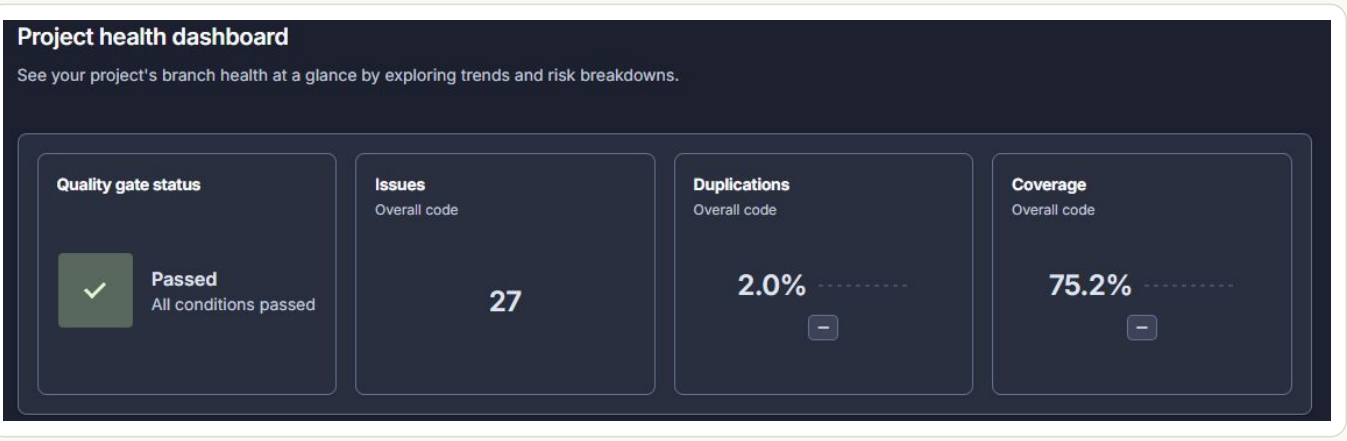


Figure 2. Tableau de bord SonarQube : état du gate et mesures globales. Les constats du projet restent visibles.

Ouvrir le tableau de bord SonarQube

Ce que prouve une CI verte

Les installations, le build, les tests, le gate et les contrôles du conteneur ont réussi pour ce commit. La disponibilité des fournisseurs IA et du transport SMTP se vérifie séparément en situation réelle.

04 Lire la couverture sans ambiguïté

La couverture indique quelles lignes mesurables ont été exécutées par les tests automatisés.

75,2 %

Couverture globale du projet

5 129 lignes exécutées sur 6 825 lignes mesurables.

Exécutées : 5 129 Non exécutées : 1 696

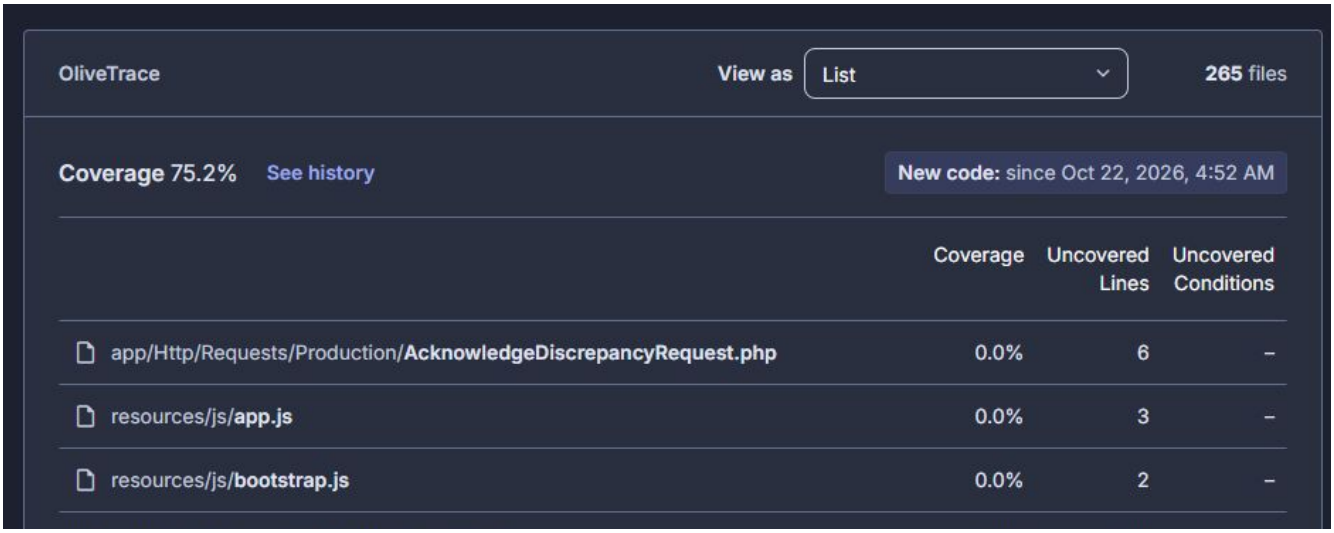


Figure 3. Vue détaillée de la couverture. Les fichiers sans exécution apparaissent avec un score de 0 %. Ce score identifie un besoin de test, pas une preuve que la fonction est défectueuse.

Explorer les fichiers et les lignes non couvertes

Pourquoi le résultat n'est pas 100 %

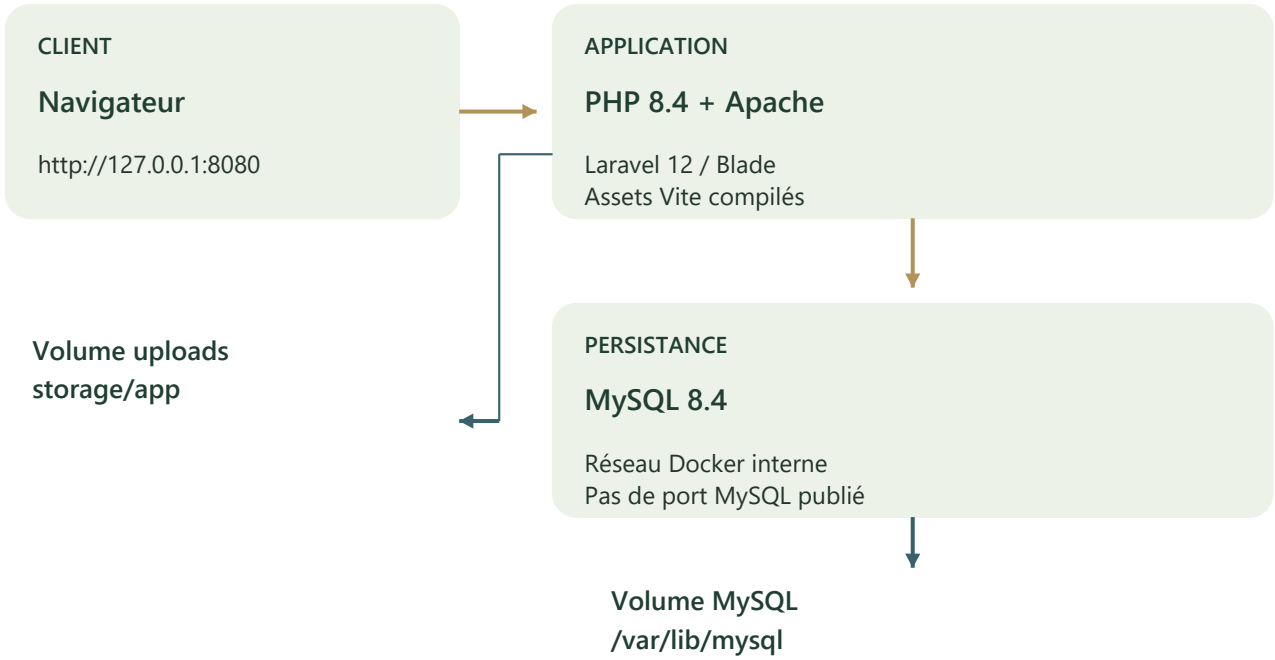
Plusieurs tests peuvent emprunter les mêmes lignes ; certains cas alternatifs, traitements ou scripts ne sont pas exercés. Ajouter des tests ciblés améliore la confiance. Même 100 % ne garantit pas l'absence de défaut.

Lecture technique. Le gate Sonar contrôle les changements depuis 284042c. Il ne remplace pas la couverture globale : son statut vert ne signifie pas que toute l'application est couverte à 100 %.

Constats globaux : 20 de fiabilité et 7 de maintenabilité à examiner ; 0 vulnérabilité détectée par les règles actives. Les factories de test sont exclues de l'analyse : leurs 17 chemins génèrent un avertissement d'import.

05 Une architecture Docker reproductible

Le même dépôt est empaqueté dans une image ; la base et les fichiers persistants sont séparés du conteneur applicatif.



Élément	Rôle dans la livraison
Build multi-stage	Node compile les assets ; PHP/Apache reçoit le code, les assets et les dépendances Composer de production.
Démarrage	Dossiers et permissions, lien public, caches Laravel, puis démarrage d'Apache.
Données persistantes	Les volumes conservent MySQL et les uploads lors de la recréation de l'application.
Exposition	Seul public/ est servi par Apache ; .env et les fichiers privés restent hors du web.

Fichiers de référence : Dockerfile · compose.yaml · script de démarrage

06 Le déploiement local est vérifié

L'application et MySQL tournent dans Docker. Ce déploiement est accessible sur le PC de démonstration.

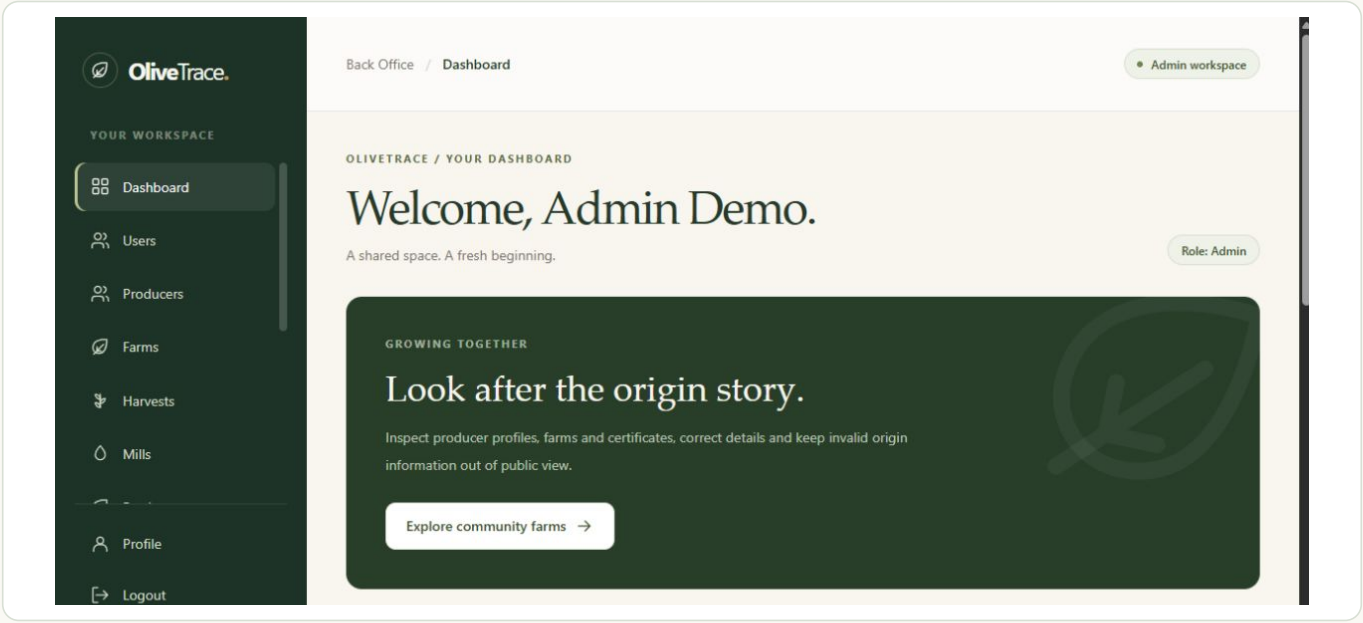


Figure 4. Connexion administrateur et tableau de bord dans l'environnement Docker local : **http://127.0.0.1:8080**.

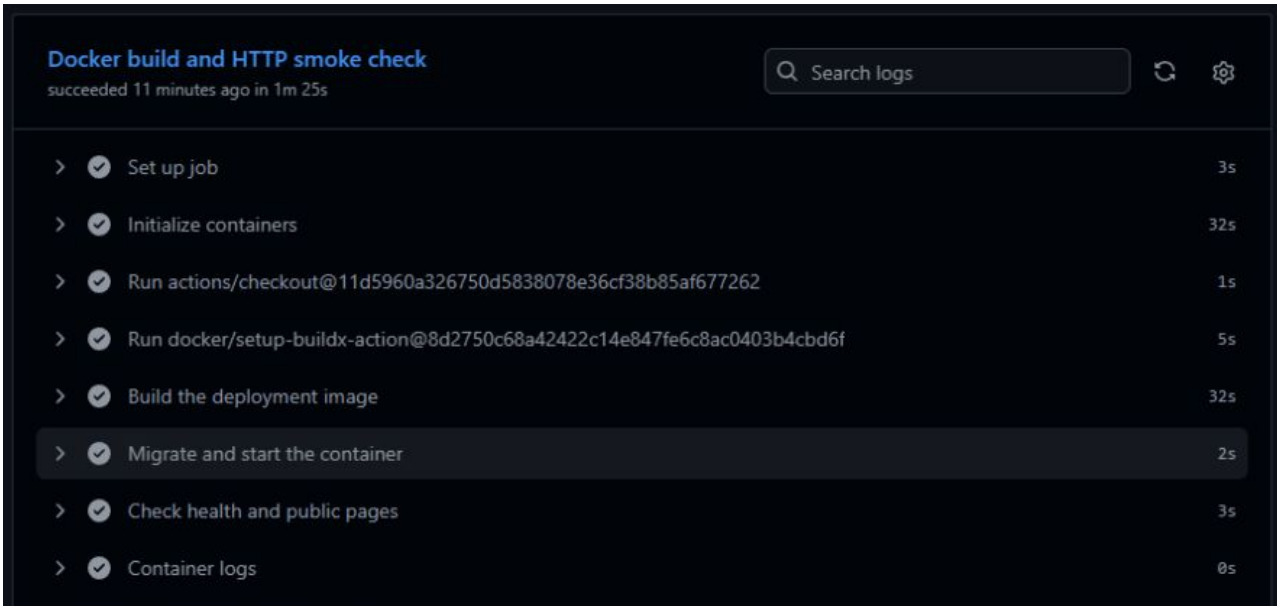


Figure 5. Job Docker réussi : construction, migration, démarrage et contrôle de santé / pages publiques.

Consulter le job et les commandes exécutées

Vérifications HTTP : /up, /, /login et /catalog retournent 200 ; /.env et un chemin de preuve privée retournent 404. Le volume d'uploads a également été vérifié après recréation de l'application.

Le serveur de développement Laravel sur :8000 est un environnement distinct. Le déploiement Docker utilise :8080 et sa propre base MySQL.

07 Railway : CD préparé, publication en attente

Aucune application publique ni URL Railway n'a encore été validée pour OliveTrace.

BLOCAGE FOURNISSEUR

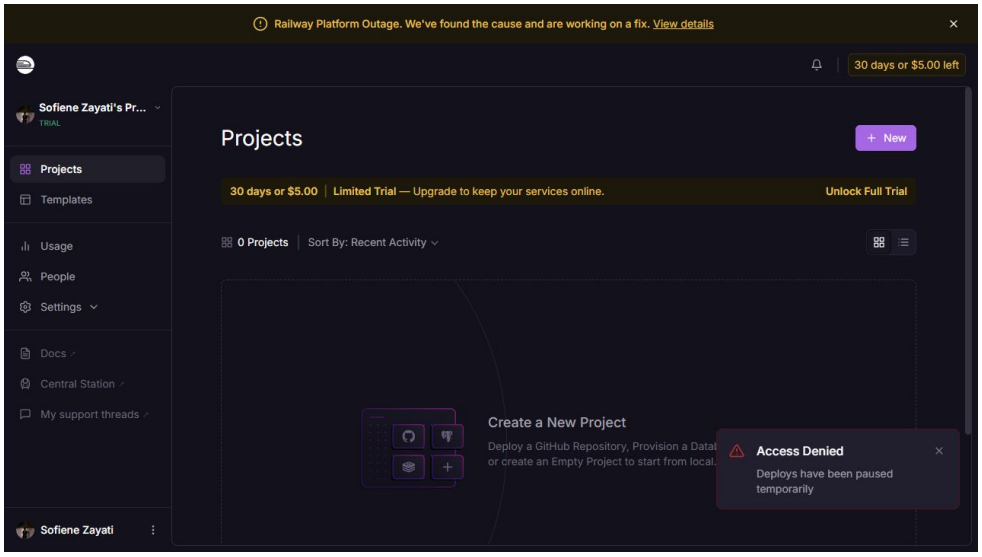


Figure 6. Refus observé pendant la préparation : « Deploys have been paused temporarily ». Le compte est en Limited Trial et aucun service OliveTrace n'a été créé.

Consulter l'incident officiel Railway

La mise à jour officielle du 22 octobre à **11 h 15 (Tunis)** indique que les builds Trial attendent encore leur restauration. La démonstration fonctionnelle reste disponible dans Docker en local.

Chaîne de CD à finaliser après rétablissement

Ordre	Action restante	Résultat attendu
1	Créer le service application et MySQL ; ajouter les volumes.	Stockage persistant et connexion à la base.
2	Connecter main, activer Wait for CI et configurer les variables privées.	Release déclenchée après une CI verte.
3	Exécuter la configuration railway.json : build, migration, /up.	Déploiement sain et domaine HTTPS.

Déjà dans le dépôt : Dockerfile, railway.json, exemple de variables, commande de migration avant déploiement et contrôle /up. **À vérifier en cloud :** première release, volumes, HTTPS, logs et intégrations externes.

08 Exploiter et rétablir le service

Les contrôles de santé et les procédures de reprise complètent les tests avant livraison.

Besoin	Mise en place / procédure
Santé de l'application	/up vérifie Laravel et une requête SQL. Une base indisponible entraîne un statut 500 sans exposer les détails de connexion.
Santé de MySQL	Le health check du service vérifie une requête SELECT 1 avant le démarrage de l'application.
Logs	Apache et Laravel écrivent vers stdout/stderr dans le conteneur ; consultation avec docker compose logs.
Persistance	Volumes séparés pour la base et storage/app. Le contrôle de recréation de l'application a conservé les uploads de test.
Secrets et droits	Variables privées hors Git ; debug désactivé en production ; contrôles de rôle/propriété et fichiers d'achat privés.
Avant une opération risquée	Sauvegarde MySQL et copie privée des uploads. Préserver APP_KEY et privilégier des migrations compatibles.

Procédure de retour à une version stable

- **1 · Diagnostiquer** - Lire les logs et contrôler /up ; interrompre la promotion d'une version défaillante.
- **2 · Revenir au code stable** - Redéployer la dernière version validée ou annuler le changement par un commit Git.
- **3 · Vérifier les données** - Un retour au code précédent n'annule pas les migrations SQL. Une restauration de données exige une sauvegarde et un contrôle adapté.

État de la supervision

Les logs et contrôles de santé sont en place. Le health check de déploiement n'est pas une surveillance continue avec alertes. Grafana et Prometheus ne sont pas installés ; la restauration complète reste une procédure à exercer.

09 Ce que la validation peut constater

Le DevOps porte sur le projet intégré ; les fonctionnalités et l'IA restent présentées par chaque membre.

Membre / module	Fonction avancée	Fonction IA
Sofiene Zayati Producteurs / fermes	Carte privée, journal de soins et tâches suivies.	Conseils de durabilité et adoption d'une recommandation en tâche.
Mariem Ammari Récolte / moulin	Workflow de livraison, trituration et écarts de quantité.	Estimation de production d'huile selon les faits de récolte.
Oussema Benjemia Laboratoire / certification	Circuit d'analyse, certificat PDF et validité.	Explication des résultats de laboratoire.
Hana Fkiri Distribution	Produits, expéditions, livraison et estimation CO2.	Aide à la description et à l'analyse de l'impact.
Aymen Jenhani Consommateurs	Réclamations/SLA, preuves privées et notifications.	Brouillons de réponse et résumés d'avis via Groq.

Socle partagé : Breeze/Blade pour l'authentification ; six rôles et comptes actifs/inactifs ; administration des utilisateurs ; validation serveur par Form Requests, policies et contraintes MySQL. Persistance Eloquent.

Parcours de démonstration proposé

- Montrer un historique / une PR, l'exécution CI verte, les rapports de test et les mesures globales SonarQube.
- Présenter le Dockerfile, les deux services et la connexion locale ; effectuer un scénario métier et un refus d'accès ou une validation serveur.
- Expliquer la chaîne Railway préparée et son blocage externe ; présenter les logs, /up et le principe de retour arrière.

Les tests IA automatisés sont simulés. Les démonstrations réelles dépendent des clés privées et de la disponibilité des fournisseurs. Les trois assistants Groq ont été contrôlés en réel ; aucun e-mail n'a été envoyé lors du contrôle SMTP.

Sources et preuves consultables

Dépôt et historique · CI et artefacts · SonarQube
Configuration de déploiement · Incident Railway
Laravel 12 : déploiement · SonarQube : qualité et périmètre